

TECHNICAL WRITE-UP: FINDESK LOAN APPLICATION PORTAL

Full Stack Engineering Technical Assessment | Inclusive FinTech Intern Track

GitHub Repository: https://github.com/ADITYA-user18/FinDesk-Loan-Application-Portal	Live App: https://fin-desk-loan-application-portal.vercel.app
Backend API: https://findesk-loan-application-portal.onrender.com	Database: Neon Serverless PostgreSQL

1. What Was Built

FinDesk is a highly polished, responsive web application designed for operations teams to manage and track borrower loan applications submitted from the field. It features:

- **Borrower Workspace:** A frictionless loan application page supporting local-language preferences (Hindi, Tamil, Telugu, Marathi) and direct clipboard reference ID copying.
- **Operations Dashboard:** Real-time statistics, advanced search, filtering by language and status, and interactive status controls (Pending, Approved, Rejected).
- **State Management & Auth:** Secure, multi-role (Borrower vs. Agent) access supported by dynamic theme toggling (Premium Black/Gold & Light modes) and custom styled form components.

2. Deployment Choices & Tech Stack

- **Frontend (Vercel):** React (Vite) + Vanilla CSS. Vercel was chosen for its sub-second page loading speeds, automated edge caching, and seamless continuous integration via Git hooks.
- **Backend (Render):** Node.js + Express API. Render offers simple build pipelines, automatic logging, and robust CORS custom request validation.
- **Database (Neon Serverless):** Serverless PostgreSQL with auto-scaling capabilities. Allowed instant setup, migration handling, and connection pooling (using *pg.Pool*) for production stability.

3. Technical Challenges & Resolutions

- **Cross-Domain Third-Party Cookie Restrictions:** Modern browsers (Safari, Chrome, iOS) block cookies across different apex domains (Vercel vs. Render). This caused a *401 Unauthorized* issue during cross-site requests.
- **Resolution:** Implemented a hybrid authentication model. The system falls back to a custom Axios interceptor that injects an *Authorization: Bearer [token]* header stored in *localStorage* if the *HttpOnly* cookie is blocked. This guarantees 100% login uptime on all laptops, phones, and incognito sessions.

4. Future Improvements

- **Voice-Based KYC:** Integrate speech-to-text libraries to allow local language names/amounts to be dictated verbally rather than typed, further reducing friction for non-literate borrowers.
- **Database Caching:** Integrate a Redis cache layer for the agent dashboard statistics to limit query loads on Neon SQL under high traffic volumes.
- **Offline Sync:** Add a service worker for progressive web app (PWA) support, enabling field agents to queue applications offline.